



كلية تكنولوجيا الصناعة والطاقة

Information Technology Department



جامعة برج العرب التكنولوجية

BORG AL ARAB TECHNOLOGICAL UNIVERSITY

INTRODUCTION TO C PROGRAMMING ESSENTIALS

First Year – Information Technology Program

spring 2026



LECTURE 4

C LANGUAGE BASICS

CONDITIONS



Dr. Dina Abdelhafiz & Dr. Ghada Fathy

CONDITIONS AND IF STATEMENTS

■ Comparison conditions

- Less than: $a < b$
- Less than or equal to: $a \leq b$
- Greater than: $a > b$
- Greater than or equal to: $a \geq b$
- Equal to: $a == b$
- Not equal to: $a != b$

THE IF STATEMENT

- Use the if statement to specify a block of code to be executed if a condition is true.

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

ONE CONDITION

```
#include <stdio.h>
```

```
int main() {  
    int x = 20;  
    int y = 18;  
  
    if (x > y) {  
        printf("x is greater than y");  
    }  
  
    return 0;  
}
```

x is greater than y

IF USING BOOLEANS

- The condition in an if statement => true or false
- Store the result in a boolean variable instead of writing the comparison directly:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int x = 20;
    int y = 18;

    bool isGreater = x > y;

    if (isGreater) {
        printf("x is greater than y");
    }

    return 0;
}
```

x is greater than y

Note: Remember to include `<stdbool.h>` when working with bool variables.

■ What will be the output?

```
#include <stdio.h>

int main() {
    int x = 5;
    if (x = 3)
        printf("Hello");
    else
        printf("Bye");

    return 0;
}
```

$x = 3$ is assignment, not comparison.

- 1- x gets assigned the value 3
- 2- The expression $x = 3$ evaluates to 3
- 3- In C, any non-zero value is true
- 4- Since $5 \neq 0 \rightarrow$ the condition is true

THE ELSE STATEMENT

- Use the else statement to specify a block of code to be executed if the condition **is false**.

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

ELSE

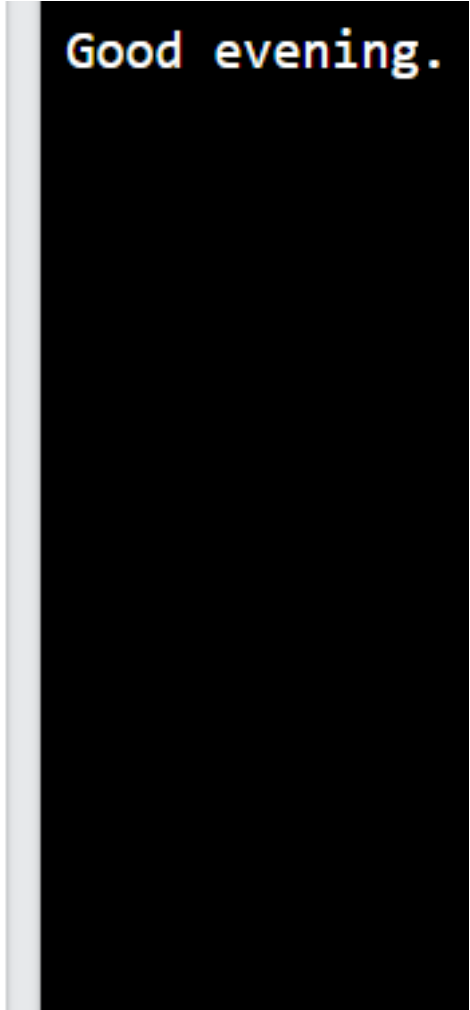
```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int time = 20;

    bool isDay = time < 18;

    if (isDay) {
        printf("Good day.");
    } else {
        printf("Good evening.");
    }

    return 0;
}
```

A terminal window with a black background and white text. The text "Good evening." is displayed on the first line. The terminal has a light gray border on the left and top sides.

Good evening.

```
#include <stdio.h>
```

```
int main() {
```

```
    int myNum = 5;
```

```
    if (myNum % 2 == 0) {
```

```
        printf("%d is even.\n", myNum);
```

```
    } else {
```

```
        printf("%d is odd.\n", myNum);
```

```
    }
```

```
    return 0;
```

```
}
```

5 is odd.

THE ELSE IF STATEMENT

- Use the else if statement to specify a new condition to test if the first condition **is false**.
- You can use else if to check **multiple conditions**, one after another.

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if condition1 is false and condition2  
    is true  
} else {  
    // block of code to be executed if both conditions are false  
}
```

EXAMPLE

```
#include <stdio.h>

int main() {
    int temperature = 30; // temperature in
    Celsius

    if (temperature < 0) {
        printf("It's freezing!\n");
    } else if (temperature < 20) {
        printf("It's cool.\n");
    } else {
        printf("It's warm.\n");
    }

    return 0;
}
```

It's warm.

EXAMPLE

```
#include <stdio.h>

int main() {
    int myNum = 10;

    if (myNum > 0) {
        printf("The value is a positive
number.");
    } else if (myNum < 0) {
        printf("The value is a negative
number.");
    } else {
        printf("The value is 0.");
    }

    return 0;
}
```

The value is a positive number.

SHORT HAND IF...ELSE (TERNARY OPERATOR)

The ternary operator *returns a value* based on a condition: if the condition is **true**, it returns the first value; otherwise, it returns the second value.

```
variable = (condition) ? expressionTrue : expressionFalse;
```

SHORT HAND IF...ELSE (TERNARY OPERATOR)

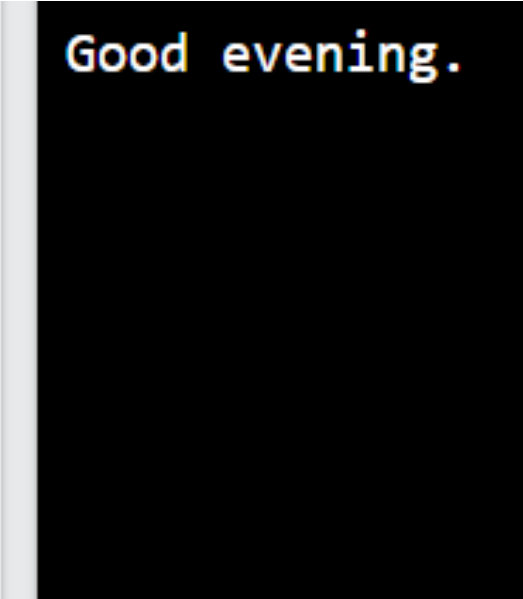
- Use Ternary operator :To replace a simple if...else statement with a single line of code

```
#include <stdio.h>

int main() {
    int time = 20;

    (time < 18) ? printf("Good day.") : printf("Good evening.");

    return 0;
}
```

A terminal window with a black background and white text displaying the output of the program: "Good evening."

Good evening.

NESTED IF

- You can also place an if statement inside another if. This is called a **nested if** statement.
- A nested if lets you check for a condition only if another condition is already true.

```
if (condition1) {  
    // code to run if condition1 is true  
    if (condition2) {  
        // code to run if both condition1 and condition2 are true  
    }  
}
```

EXAMPLE

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int age = 20;
    bool isCitizen = true;

    if (age >= 18) {
        printf("Old enough to vote.\n");

        if (isCitizen) {
            printf("And you are a citizen, so you can
vote!\n");
        } else {
            printf("But you must be a citizen to vote.\n");
        }
    } else {
        printf("Not old enough to vote.\n");
    }

    return 0;
}
```

```
Old enough to vote.
And you are a citizen, so you can vote!
```

LOGICAL OPERATORS IN CONDITIONS

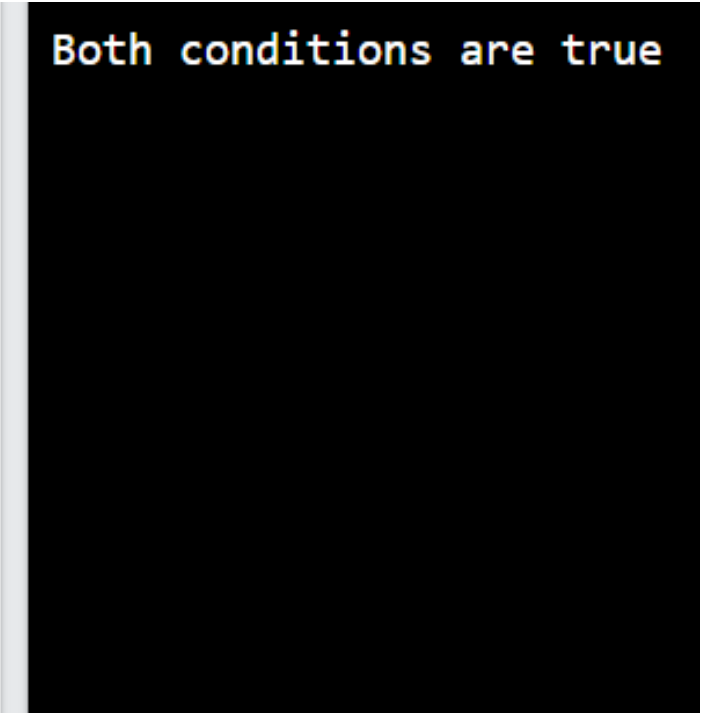
- && (AND) - all conditions must be true
- || (OR) - at least one condition must be true
- ! (NOT) - reverses a condition (true → false, false → true)

```
#include <stdio.h>

int main() {
    int a = 200;
    int b = 33;
    int c = 500;

    if (a > b && c > a) {
        printf("Both conditions are true\n");
    }

    return 0;
}
```

A screenshot of a terminal window with a black background. The text "Both conditions are true" is displayed in a light blue, monospaced font, matching the output of the C program shown on the left.

Both conditions are true

SWITCH STATEMENT

- Instead of writing **many** if..else statements, you can use the switch statement.
- The switch statement selects one of many code blocks to be executed:

```
switch (expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block  
}
```

EXAMPLE

- A customer wants to order **Coffee**.

```
#include <stdio.h>

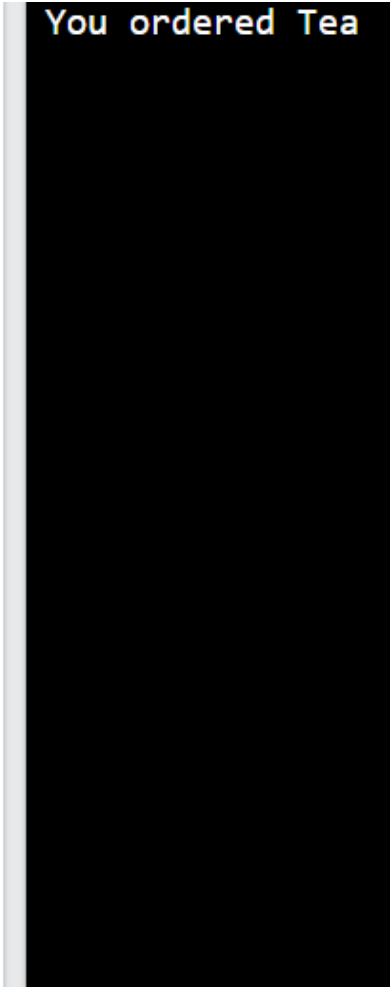
int main() {
    int choice = 2;

    // 1 = Coffee
    // 2 = Tea

    switch (choice) {
        case 1:
            printf("You ordered Coffee");
            break;

        case 2:
            printf("You ordered Tea");
            break;
        default:
            printf("Invalid choice");
    }

    return 0;
}
```



You ordered Tea

BREAK STATEMENT

- What is the purpose of the **break statement** in switch?:Ends the switch block
- What will be the output?

```
#include <stdio.h>

int main() {
    int x = 2;
    switch(x)
    {
        case 1: printf("One");
        case 2: printf("Two");
        case 3: printf("Three");
    }

    return 0;
}
```

Output
Two
Three

- **Which data types can be used inside a switch expression in C?**
- A) int and char only
- B) float only
- C) double only
- D) All data types

What is the purpose of the **switch** statement in C?

- A. To select one of many code blocks to be executed
- B. To loop through a set of conditions
- C. To compare two values
- D. To declare multiple variables

Which of the following operators checks if two values are equal in C?

- A. =
- B. ==
- C. !=
- D. <>

What is a boolean data type in C used for?

- A. Storing floating-point numbers
- B. Storing whole numbers
- C. Storing true or false values
- D. Storing characters

What is the purpose of the **else** statement in C?

- A. To execute a block of code if the `if` condition is true
- B. To execute a block of code if the `if` condition is false
- C. To create a new condition
- D. To end the program

What is the purpose of the ternary operator in C?

- A. To write multiple lines of code within an if...else statement
- B. To replace a simple if...else statement with a single line of code
- C. To handle complex conditions that require nested if statements
- D. To declare a new variable

■ What will be the output?

```
#include <stdio.h>

int main() {
    int x = 4;
    if (x % 2 == 0)
        printf("Even");
    if (x > 2)
        printf("Greater");

    return 0;
}
```

```
#include <stdio.h>

int main() {
    int x = -1;
    if (x)
        printf("True");
    else
        printf("False");

    return 0;
}
```



Happy Coding!